

EBU6304 Software Engineering Group Project

2025-26

First report

Group number	27	
member 1	QM no: 231222017	Name: Yifu Feng
member 2	QM no: 231222729	Name: Yuzhang Wu
member 3	QM no: 231220806	Name: Chensiyuan Qing
member 4	QM no: 231221054	Name: Fuhe Huang
member 5	QM no: 231221582	Name: Tao Li
member 6	QM no: 231220297	Name: Mu Du

Requirements

This report explains how Group 27 developed the requirements for the TA Recruitment System and, more importantly, how those requirements were progressively translated into a product backlog and then into a prototype structure. The revised version places the emphasis on process logic. Rather than discussing requirements, backlog and prototype as three separate artefacts, it presents them as stages in a single chain of refinement: first the team identified stakeholder needs, then it reformulated those needs as implementable stories, and finally it adjusted the prototype so that the interface reflected the revised backlog. This makes the development process easier to follow and better demonstrates how the team worked iteratively.

1. Requirement discovery through evidence

The team did not begin by sketching pages or inventing features. It began with fact-finding. The project brief, lecture guidance, the requirements workshop, the role-based discussion of Applicant, Module Organiser and Admin needs, and the follow-up interview with an experienced TA were used together to establish what problem the system was actually expected to solve. This starting point matters because it shaped the whole project: the team approached the system as a response to a fragmented recruitment process rather than as an abstract software idea.

The workshop on 10 March 2026 provided the first shared understanding of stakeholder needs. From the applicant perspective, the main concerns were repeated data entry, insufficient vacancy information before application, the ability to apply for more than one role, and weak visibility after submission. From the MO perspective, the core issue was the inefficiency of reviewing applicants across disconnected records. From the admin perspective, the priority was not advanced analytics but a practical way to see accepted workload and spot possible overload or conflict cases. At this stage the requirements were still broad, but they were already specific enough to define the project boundary.

The follow-up interview on 13 March 2026 was especially important because it forced the team to validate and narrow those early ideas. It confirmed that module-specific skills matter during screening, that successful applicants are commonly informed by email, that detailed rejection explanations are not usually central to the process, and that tutorial scheduling normally happens after recruitment rather than inside it. These findings did not simply add more detail; they also removed unnecessary scope. As a result, the team could justify a first release focused on recruitment itself, while keeping automated email, detailed rejection workflows and post-recruitment scheduling outside the early version. (see Supporting Material A)

2. Translating requirements into backlog structure

Once the team had a clearer understanding of stakeholder needs, the next task was to convert those needs into backlog items that were small enough to implement, test and discuss. This was not a direct copy from interview notes into user stories. The team first reorganised the requirements into broader functional areas, then broke each

area into smaller actions that could be prioritised and eventually represented in the interface. In practice, this meant turning problem statements into structured stories with clear purposes and acceptance criteria.

That restructuring can be seen in the final backlog organisation. Requirements were grouped into three main areas — Applicant Profile and Application, Vacancy Posting and Applicant Review, and Admin Workload Monitoring — together with cross-cutting reliability stories. A broad requirement such as 'applicants need a clearer and less repetitive process' therefore became several distinct backlog items with different roles in the system: US01 reusable applicant profile, US02 CV upload, US03 browse vacancies, US04 vacancy details, US05 apply for vacancy, US06 apply for multiple roles and US07 application status. Similarly, the organiser requirement of 'review applicants efficiently' was decomposed into vacancy creation, applicant-list viewing, profile-and-CV review, decision recording, workload visibility and review notes.

The backlog therefore became the team's working interpretation of the requirements. It is important that this happened through refinement rather than through a one-off rewrite. The follow-up interview and subsequent discussion led to changes in emphasis inside the backlog itself. Some items were strengthened because the evidence clearly supported them, such as module-specific vacancy information, status consistency and workload visibility. Other ideas were deliberately weakened or postponed because they were not justified strongly enough for the first version. In this sense, the backlog was not simply a planning list; it was the mechanism through which raw requirements were transformed into a more disciplined system definition. (see Supporting Material B)

This refinement process is evidenced in Supporting Material B.

Member	Main responsibility	How this contribution helped the requirement-backlog transition
Yuzhang Wu	requirements and evidence	Recorded workshop and interview outputs and helped turn raw stakeholder findings into a usable basis for story drafting.
Chensiyuan Qing	backlog and prioritisation	Converted broad stakeholder needs into implementable stories, priorities and acceptance criteria.
Fuhe Huang	prototype and UX	Checked whether the refined stories could be represented clearly through page purpose, flow and interface structure.
Tao Li	report integration	Helped keep requirement wording, backlog wording and prototype copy aligned and internally consistent.
Mu Du	testing and reliability	Brought validation, duplication control and consistency concerns into the backlog instead of leaving them implicit.
Yifu Feng	End-to-End Backlog Transition Ownership	Led end-to-end requirements-to-backlog transition, made final scope/priority decisions, and ensured backlog alignment with business and stakeholder value

This transition from requirements to backlog was also a collaborative process rather than a sequence of isolated handovers. The workshop outputs recorded by Yuzhang Wu were not treated as finished requirements by themselves; instead, they were discussed further with Chensiyuan Qing when stories were being defined, with Mu Du when validation and reliability implications became clearer, and with Yifu Feng when the team needed to decide which ideas should remain inside the first release. In parallel, Fuhe Huang and Tao Li helped ensure that the same requirement language could later be carried into page structure and prototype copy without changing its meaning. As a result, requirement interpretation, backlog drafting and wording refinement happened through repeated checking across roles rather than through one member writing independently and the others simply accepting the result.

This pattern of collaboration mattered because many of the most useful backlog decisions depended on negotiation between perspectives. Applicant-side needs had to be balanced against organiser workflow, quality constraints had to be brought into what might otherwise have become a purely functional backlog, and prototype feasibility had to be considered before story wording became fixed. The team therefore used collaboration not only to divide labour, but also to test whether a requirement still made sense once it was expressed as a story and then imagined as part of a working interface.

3. Prioritisation and estimation as the mechanism of selection

Once the requirements had been translated into stories, prioritisation and estimation were used to decide what belonged to the first release and what should wait. MoSCoW prioritisation was particularly useful because it prevented the team from treating every stakeholder wish as equally urgent. Instead, it forced a distinction between the minimum viable recruitment flow and features that would improve the system but were not essential for the first version.

This is why stories such as US03, US04, US05, US09, US10, US11, US12, US14 and US17 were treated as Must items. Without them, the system would not support the core route from vacancy publication to application, review, outcome and workload visibility. By contrast, stories such as US15 filter workload records, US16 highlight overload or conflict, US21 review notes, US22 optional feedback and US23 applicant count remain valuable, but they refine the experience rather than define it. Their placement as Should items reflects the team's effort to keep the first release realistic while still leaving room for later improvement.

Story points then performed a second role: they translated the prioritised backlog into something that could support implementation planning. The estimates in the backlog reflect complexity, coupling and dependency rather than the number of visible interface elements. For example, US04 view vacancy details is only 2 points because, once vacancy data exists, the story is mainly about structured presentation. By contrast, US09 create and publish vacancy and US12 record offer or rejection outcome are 5-point stories because they combine validation, state change and downstream effects across other pages. In this way, prioritisation answered the question of what mattered first, while estimation answered the question of what would take more coordination to deliver.

4. Translating the backlog into prototype changes

The most important improvement in the later drafting stage was that the prototype stopped being treated as an independent design task and became a direct expression of the backlog. In earlier discussion it would have been easy to produce generic applicant, organiser and admin screens with only superficial links to requirements. Once the backlog became more specific, however, the prototype had to change. Page purpose, information layout and even navigation were revised so that each screen corresponded to refined stories rather than to vague role labels.

The clearest example is the applicant vacancy flow. When the requirements were still broad, a vacancy page might only have needed a title, short description and an Apply button. After backlog refinement, that was no longer enough. US03, US04 and US05 made it clear that applicants needed module-specific information, required skills, workload, deadline and a visible route into application. The prototype therefore changed accordingly: P05 and P06 no longer function as generic job-list pages, but as decision-support pages that expose the exact information applicants need before committing to an application.

The second example is the application-status flow. The requirement findings showed that applicants wanted more transparency after submission, but the follow-up interview also showed that a detailed personalised rejection process was not realistic for the first release. The backlog translated that requirement into US07 view application status and the lighter-weight US22 optional outcome feedback. The prototype then adapted by making P07 a clear status page that uses standard labels such as Submitted, Offered and Unsuccessful, with short feedback only where appropriate. This is a good example of how backlog refinement directly shaped the final page design.

The organiser review flow provides an even stronger illustration of the logic. The original requirement was simply that organisers should be able to review applicants efficiently. In backlog form, that broad need was split into US10 view applicants for a vacancy, US11 review applicant profile and CV, US12 record offer or rejection outcome, US13 view workload when needed, and US21 add review notes. Because the backlog separated these actions, the prototype also had to become more structured. P10 was adjusted to show profile preview, CV access, workload summary, notes and decision actions in the same review space. In other words, the prototype became more detailed because the backlog became more precise.

A similar pattern appears in the admin and quality pages. The refined backlog clarified that admin users mainly needed a practical overview of accepted assignments rather than a full scheduling system. This is why US14, US15 and US16 focus on overview, filtering and overload flags rather than timetable building, and why P11 reflects exactly that narrower purpose. The same logic applies to quality stories: once US17 and US18 made validation, duplicate prevention and inconsistency handling explicit backlog items, P12 had to be adjusted to show warnings, validation messages and error states rather than only normal success flows.

Overall, the prototype became more coherent because the backlog became more specific. Changes in backlog wording were reflected in page content, page purpose and navigation logic. This is also why the final prototype uses consistent role names and consistent status terms across multiple pages. Once the backlog introduced cross-page concerns such as traceability, consistency and workload visibility, the prototype had to reflect them visibly. The prototype was therefore not designed first and justified later; it was revised as the backlog evolved.

A more detailed mapping between requirement insight, backlog expression and prototype response is shown in Supporting Material D.

This stage also depended on close coordination across the team. Prototype adjustment was not simply a visual translation carried out after the backlog was finished; it required repeated checking between those refining stories and those shaping page flow. As backlog wording became more precise, prototype structure had to be reviewed again so that page content, labels and actions continued to match the agreed stories. This is why the final prototype reads as a coordinated system rather than as a set of loosely related screens.

Requirement insight	Backlog expression	Prototype response	Why this change improved coherence
Applicants need clearer vacancy information before applying.	US03, US04, US05	P05/P06 were revised to show module, required skills, workload, deadline and the application route.	The applicant pages became decision-support pages rather than generic listings.
Applicants want more transparency after submission, but not a complex rejection workflow.	US07, US22	P07 was adjusted to use standard status labels and optional short feedback only where appropriate.	The status page now reflects realistic scope rather than promising an overdesigned process.
Organisers need review information in one place.	US10, US11, US12, US13, US21	P10 was revised to combine profile preview, CV access, workload summary, notes and decision actions.	One broad organiser requirement became a structured review workflow.
Admin needs practical workload visibility, not full scheduling.	US14, US15, US16	P11 was narrowed to accepted jobs, filtering and overload flags.	The admin prototype now follows the refined backlog instead of expanding beyond scope.
Reliability and validation must be visible requirements.	US17, US18	P12 was adjusted to show warnings, validation messages and error states.	Quality requirements became demonstrable design elements rather than hidden assumptions.

5. Building the iteration plan from the aligned backlog and prototype

The iteration plan became much easier to justify once requirements, backlog and prototype had been aligned. To ensure feasibility and alignment with the defined scope, the team selected a lightweight Java Servlet/JSP web-based application. This approach was sufficient to support key functionalities such as role-based interfaces, application submission, status tracking, validation, and workload management, without introducing unnecessary complexity. As a result, the technical choice reinforced the MVP orientation of the project and supported a structured and achievable iteration plan. Sprint 1 could no longer be described vaguely as the stage where the team 'did requirements'. Instead, it became the phase in which the technical route was fixed, evidence was gathered, the backlog was drafted and revised, and the prototype was adjusted so that it matched the revised story structure. In other words, Sprint 1 produced the logic that made later implementation possible. (see Supporting Material F)

Sprint 2 then follows naturally from that aligned structure. Once the applicant profile, vacancy information and page flow are clearly defined, the team can build the main recruitment route: application submission, multiple-role support, application status, vacancy posting, applicant-list review, profile-and-CV review, and offer or rejection outcomes. Because the prototype already reflects these stories, Sprint 2 is not just a collection of unrelated tasks. It is the implementation of the now-stabilised core workflow.

Sprint 3 becomes even easier to explain when seen as a continuation of backlog refinement. The stories assigned here — workload filtering, overload highlighting, duplicate or inconsistent data rejection, status consistency, traceability, review notes, optional feedback and applicant counts — are not random extras. They are the backlog items that depend on the existence of the core workflow and therefore make more sense once the main path has been implemented and represented in the prototype. The plan is therefore sequential in a logical sense, not just chronological.

Seen in this way, the iteration plan represents the final stage in a chain of translation. Fact-finding generated raw stakeholder needs; backlog refinement converted those needs into implementable stories; prototype adjustment made those stories visible in interface form; and the sprint plan then ordered those stories into a realistic sequence of work. This is why the main body reads more coherently when Sections 4 and 5 are tied to the requirement-to-backlog-to-prototype logic rather than being discussed as isolated planning sections.

Conclusion

The revised main body shows that Group 27 did not move directly from vague needs to finished screens. Instead, the team used evidence to identify requirements, used the backlog to break those requirements into prioritised and estimated stories, and then used the evolving backlog to adjust what the prototype needed to show. Only after this alignment did the iteration plan become fully convincing. This makes the project process more credible because it shows that the backlog and prototype were both products of requirement refinement rather than parallel documents written independently.

Supporting Materials

The following supporting materials provide evidence for the process claims made in the main report. They are presented separately so that the five-page report remains concise while the underlying decision trail, artefact evolution and collaboration record remain visible.

Supporting Material A. Evidence to Requirement and Scope Decisions

Evidence source	Observed need / issue	Team interpretation	Resulting decision
Requirements workshop	Applicants repeated the same information across applications and lacked clear vacancy detail before deciding to apply.	The first release needed a clearer applicant-side flow with reusable profile data and decision-support information.	Translate the applicant requirement into separate but related stories for profile reuse, CV upload, browsing, vacancy details, application and status visibility.
Role-based discussion	MOs found applicant review inefficient when information was scattered across records and pages.	Review work should be treated as a structured workflow rather than a single generic review screen.	Create distinct stories for vacancy publishing, applicant list review, profile/CV review, decision recording and workload visibility.
Follow-up TA interview	Detailed rejection explanations and tutorial scheduling were not central to an early recruitment release.	Not every plausible feature belonged in the MVP.	Keep detailed rejection workflow and post-recruitment scheduling out of the first release.
Internal scope review	Advanced ideas such as AI matching and extended balancing risked inflating the system beyond a realistic first iteration.	The team needed to preserve a manageable MVP.	Deprioritise advanced intelligent-support features and focus on the core recruitment route.

This table supports Section 1 of the main report by showing that requirements were derived through evidence and then narrowed through explicit scope decisions, rather than expanded without control.

Supporting Material B. Backlog Evolution Across Three Versions

Only the most representative changes are shown below. The purpose is not to reproduce the entire backlog, but to demonstrate how discussion and stakeholder feedback altered specific stories across iterations.

Feature / story	Version 1	Version 2	Version 3	Why the change mattered
US04 Browse vacancies / view details	Applicant-side browsing existed as part of the initial flow.	Browsing remained, but required login and was treated more conservatively within the refined flow.	The story was explicitly extended to browsing available vacancies before login.	This is the clearest evidence that accessibility and entry into the system became a stronger design concern in the

Feature / story	Version 1	Version 2	Version 3	Why the change mattered
				later iteration.
Applicant count visibility	Applicant count was initially shown as a transparency feature.	The feature was removed from the visible design and lowered in priority after the team observed that many real recruitment systems do not display it.	The feature was reintroduced because it became an explicit client requirement, although its priority remained controlled.	This shows that refinement was not simply additive: features could be introduced, removed and reintroduced for different reasons.
Admin workload scope	Admin concerns were recognised broadly.	Workload-related functionality was narrowed towards accepted assignments and manageable controls.	The structure stabilised around practical workload visibility rather than broad administrative expansion.	This demonstrates scope control and a move away from over-design.
Advanced intelligent-support ideas	Ideas such as matching or balancing were discussed conceptually.	They were pushed down the backlog as non-essential for the first release.	They remained low priority in the final planning view.	The team preserved feasibility by distinguishing desirable extensions from MVP essentials.

Supporting Material B supports Sections 2 and 3 of the main report by showing that backlog refinement involved selection, weakening, postponement and justified reintroduction rather than one-directional feature growth.

Supporting Material C. Selected Prototype Changes that Evidence Refinement

The report does not need the full prototype repeated here. Instead, the three most important changes are highlighted to show how backlog decisions altered the interface.

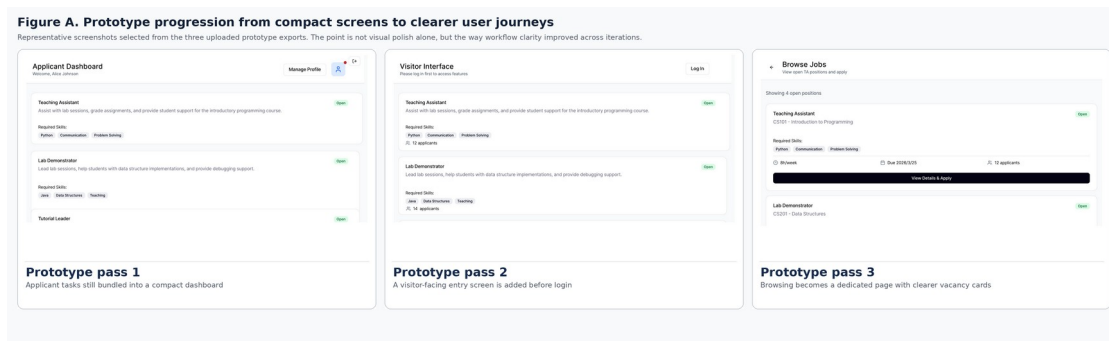


Figure C1. Simplification from Version 1 to Version 2

The first major revision reduced unnecessary page fragmentation and made the applicant flow more compact. This supports the report's claim that improvement was not measured by adding screens, but by making the workflow easier to follow.

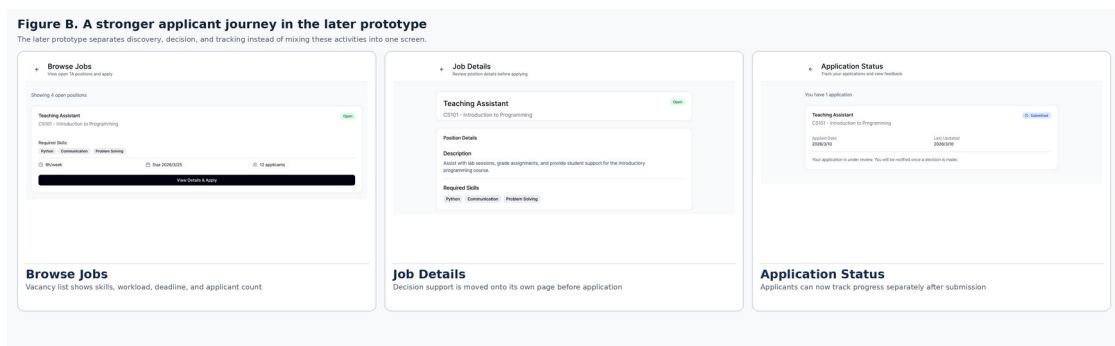


Figure C2. Applicant entry and browsing refinement

The later prototype introduces a clearer browsing route and supports the revised form of US04, where applicants can inspect opportunities before login. This demonstrates a direct connection between story refinement and interface entry structure.

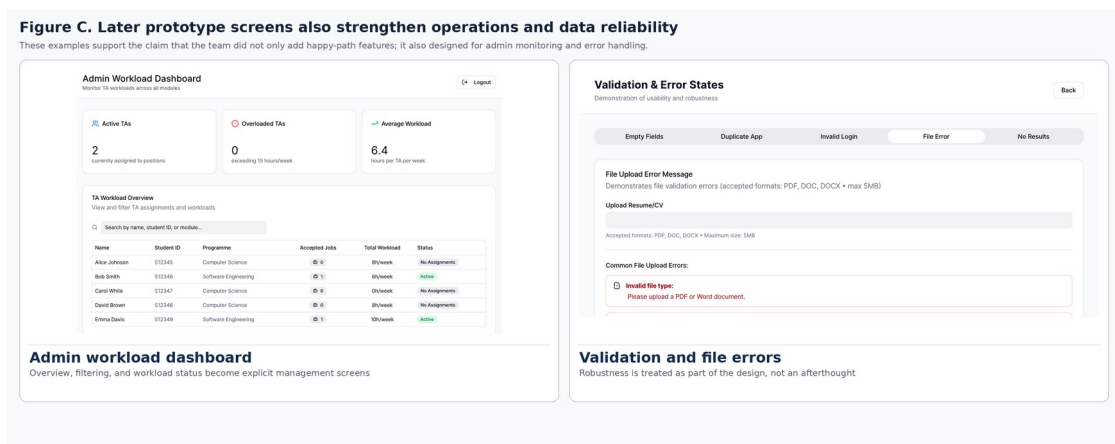


Figure C3. Quality and management changes

Validation visibility and workload-oriented pages became more explicit as reliability and admin scope were clarified in the backlog. The screenshots therefore support the argument that quality requirements were made visible rather than left implicit.

Supporting Material D. Requirement → Backlog → Prototype Traceability

Requirement insight	Backlog expression	Prototype response	Why this evidence matters
Applicants need enough vacancy detail to make an informed decision.	US03 browse vacancies, US04 view vacancy details, US05 apply for vacancy.	Job-list and vacancy-detail pages were revised to expose module, skills, workload, deadline and the route into application.	This shows that a broad requirement became a structured decision-support interface.
Applicants want transparency after submission, but the first release should avoid overdesigned rejection handling.	US07 application status and lighter-weight outcome feedback.	Status pages use concise labels such as Submitted, Offered and Unsuccessful rather than a heavy rejection workflow.	This demonstrates realistic scoping rather than feature inflation.

Requirement insight	Backlog expression	Prototype response	Why this evidence matters
Organisers need one coherent review workflow.	US10, US11, US12, US13 and US21 split a broad organiser need into visible review actions.	The organiser review page combines profile preview, CV access, notes, workload summary and decisions.	This evidences how decomposition in the backlog led to more structured prototype pages.
Admin needs practical workload visibility rather than full scheduling.	US14, US15 and US16 centre on overview, filtering and overload/conflict signalling.	The admin/workload page was narrowed accordingly.	This shows that prototype scope followed backlog discipline rather than expanding on its own.

Supporting Material E. Collaboration as a Driver of Artefact Change

Situation	Discussion trigger	Collaborative action	Artefact change
Applicant count was questioned.	Comparison with common recruitment systems suggested that transparency of this kind is not always exposed.	The team discussed realism versus perceived usefulness and agreed to remove the feature and lower its priority in the second version.	Backlog V2 and prototype V2 both reflected the removal.
Applicant count returned.	The client later made this an explicit requirement.	The team reintroduced the feature without allowing it to dominate the overall MVP.	Backlog V3 and prototype V3 reflected the reintroduction, but priority remained controlled.
Browsing before login was added.	Stakeholder understanding of user entry behaviour improved.	The team refined US04 and aligned page flow accordingly.	Backlog V3 and the later prototype introduced pre-login browsing.
Validation became explicit.	Reliability and duplication concerns were raised during testing-oriented discussion.	Validation and inconsistency handling were converted into explicit stories rather than hidden assumptions.	Backlog and prototype both gained visible error and warning logic.



Figure E1. Team discussion session during backlog refinement and requirement analysis.

This figure provides evidence of collaborative decision-making within the team. During this session, requirements and backlog items were discussed and refined collectively. Key changes, including the refinement of US04 (introduction of pre-login browsing) and the removal and later reintroduction of the applicant count feature, emerged from such discussions. These interactions directly influenced backlog evolution and subsequent prototype modifications. Supporting Material E therefore reinforces the argument that teamwork contributed to artefact change rather than merely task distribution.

Supporting Material F. Story Clusters, Relative Estimation and Sprint Logic

Story cluster	Relative effort logic	Sprint	Reason for placement
Login, profile reuse, basic browsing and vacancy detail visibility	Lower-complexity presentation and navigation work, but foundational to later tasks.	Sprint 1	These items establish the user entry route and the information structure required by the rest of the system.
Application submission, multiple-role support, application status, vacancy publishing and applicant review	More coordination-heavy because they combine validation, state change and cross-page dependency.	Sprint 2	These stories form the core recruitment workflow and therefore define the central implementation phase.
Workload filtering, overload flags, duplicate/inconsistent data handling, review notes, optional feedback and applicant counts	Mostly dependent or refinement-oriented stories that build on the core workflow.	Sprint 3	These features make more sense once the main path already exists and can therefore be layered on in a controlled way.

Supporting Material F supports the report's estimation and iteration-planning claims by showing that relative effort and dependency were used together, rather than distributing stories evenly across time.